Chapter 1

Data Types

In programming, we work with different kinds of information, or data. A *data type* is a way of classifying this information, typically stored as a variable, so that the computer knows how to handle it. Just as different objects in everyday life serve different purposes, various kinds of data are handled differently in a program. For instance, numbers are often used in calculations, while text is treated as characters or words.

Think of data types like containers in your kitchen. If you want to store soup, you'd use a bowl, while juice would go in a glass. Similarly, in programming, you need to choose the right data type to store and process the information correctly. Using the right "container" helps ensure that the computer knows what to do with your data.

In MATLAB, understanding data types is essential not just for organizing your data, but also for interacting with the built-in tools and functions MATLAB offers. These tools use a variety of data types as both input and output. By knowing how to work with different data types, you can ensure that your programs communicate effectively with MATLAB's powerful built-in functionality.

In this chapter, we'll explore most of the different data types in MATLAB, when to use them, and how this knowledge will help you interface better with MATLAB's capabilities.

1.1 Numbers

MATLAB, short for "matrix laboratory," was designed specifically for working with matrices, arrays, and vectors. In MATLAB, the fundamental data type for handling numerical values is the *array*. This means that even single numbers (scalars), vectors, and matrices are all represented as arrays. We can even have empty arrays. Understanding how MATLAB defines and uses these different types of arrays is essential to working efficiently with the software.

Here's how MATLAB classifies these key terms:

Scalar: An array with a single element.

Vector: A two-dimensional array that has either 1-by-N (a row vector) or N-by-1 (a column vector) dimensions.

Matrix: A two-dimensional array with more than one element in both dimensions.

Array: A general term in MATLAB that can refer to data with more than two dimensions.

By treating everything as an array, MATLAB simplifies many operations, making it easy to apply the same functions to scalars, vectors, matrices, and higher-dimensional arrays.

The default type of number in a numeric array *double*, short for double-precision floating-point, and is suitable for most calculations due to its high precision and wide range. When you enter a number in MATLAB, it is automatically stored as a double, allowing for efficient handling of both integers and real numbers, including very large or very small values.

However, other numeric data types are important to understand, especially when interfacing with MATLAB utilities and libraries that expect specific types of input or output:

- **Integers** (e.g., `int8`, `int16`, `int32`, `int64`): These types store whole numbers and are more memory-efficient when decimal values aren't needed. For example, when counting items or processing digital signals, integers can save memory and improve performance.
- **Unsigned Integers** (e.g., `uint8`, `uint16`): These represent non-negative whole numbers and are commonly used in tasks such as image processing, where pixel values are naturally non-negative and often fall within a specific range (e.g., 0 to 255 for `uint8`).

You may encounter two special numeric values: `NaN` and `Inf`. These values arise in situations where the result of a calculation doesn't produce a standard number and can be indicators of a bug.

- `NaN` **(Not a Number):** This value represents undefined or unrepresentable results, such as the outcome of `0/0` or the square root of a negative number. `NaN` acts as a placeholder for invalid or missing numerical data and can propagate through calculations, indicating that the result is indeterminate.
- `Inf` **(Infinity):** This value occurs when a calculation produces a result too large to represent, such as division by zero (e.g., `1/0`). MATLAB distinguishes between positive (`Inf`) and negative infinity (`-Inf`). These values are often encountered in cases where numerical limits are exceeded, such as in iterative processes or extremely large datasets.

1.2 Strings

Under the hood, computer programs work with numbers, but people are often better at processing language. To bridge this gap MATLAB uses arrays of *characters* and arrays of *strings*.

A character array is a one-dimensional sequence where each element is a `char` type, analogous to the vectors we saw in Chapter 4 where each element was a number. We can create a character array using a single quote, `'`, and use some of the same techniques to determine the size and access elements via indexing.

```
>> charray = 'The quick brown fox';
>> size(charray)
ans =
     1    19
>> charray(5:9)
ans =
    'quick'
>> charray(end-3:end)
ans =
    ' fox'
```

MATLAB uses a standard encoding to represent each character as a number (actually a 16-bit unsigned integer). We can peer under the hood by converting the array of charters to an array of the codes:

```
>> charray = 'hello!';
>> double(charray)
ans =
   104   101   108   108   111    33
```

A `string` is a more powerful data type for storing text that simplifies manipulating text using functions such as `contains`, `replace` and `split`. You can create a string using double quotes, ". Strings, like numbers in MATLAB, are stored in arrays, so a single string is actually a 1-by-1 array of strings.

```
>> str = "jumps over the lazy dog";
>> size(str)
ans =
     1     1
```

Because a string element is a single object, we can't use the same numerical indexing syntax, but MATLAB includes a function to support access by location:

```
>> extractBetween(str, 16, 19)
ans =
    "lazy"
```

We can use addition to concatenate strings, but there is no sense of subtraction

```
>> hw = "hello" + " world!"
hw =
    "hello world!"

>> h_w = "hello" - " world!"
Operator '-' is not supported for operands of type 'string'.
Error in types_examples (line 19)
h_w = "hello" - " world!"
```

1.2.1 Creating Strings

So far our examples have typically generated output to the Command Window in raw form. We'll run into many situations where we'd like to improve the formatting of that text output, mixing numbers and letters. This is particularly helpful when recording/logging data to a text file.

Two commands generate formatted strings: `sprintf` which returns a formatted string and `fprintf` which writes that string to the screen or a file. For both commands we provide a *format string* which contains the instructions for how to present the output. Within the format string we can include *special characters* that can't be included as ordinary text special two-character codes such as `\n` for a new line and `\t` for a tab.

```
>> table_str = sprintf(" cats \t dogs \n salt \t pepper")
table_str =
    " cats       dogs
      salt       pepper"

>> disp(table_str);
 cats    dogs
 salt    pepper

>> fprintf(" yin \t yang \n fish \t chips \n")
 yin     yang
 fish    chips
```

The format string can also include *format specifiers* which are then replaced by numbers or characters in the specified format. Format specifiers start with a percent sign, `%`, and follow this prototype:

```
%[identifier][flags][width][.precision][subtype]specifier
```

The items within brackets are optional. The *specifier* indicates the format of the provided data. Common specifiers are `%f` for fixed point numbers, `%d` for integers, `%E` for exponential (scientific) notation and `%s` for strings.

```
>> % Fixed-point
>> fprintf("pi = %f\n", pi);
pi = 3.141593
```

Notice that we added a newline character at the end of the format string, otherwise the next command prompt is on the same line as the output.

We can be more explicit by adding *width* and *precision* items to our format specifier:

```
>> fprintf("pi also equals %4.2f\n", pi)
    pi also equals 3.14
```

We can provide multiple specifiers and values, which MATLAB uses in order:

```
>> fprintf("There are %d %s and %d %s\n", ...
3, "apples", 5, "oranges");
    There are 3 apples and 5 oranges
```

And we can provide the arrays as values:

```
>> fprintf("Random numbers: %5.3f\n", rand(1,3))
Random numbers: 0.766
Random numbers: 0.795
Random numbers: 0.187
>> fprintf("Random numbers: %5.3f, %5.3E, %.3f \n", rand(1,3))
Random numbers: 0.490, 4.456E-01, 0.646
```

1.2.2 Analyzing Strings

MATLAB also includes a number of ways to examine strings and take them apart. These can be helpful for enabling MATLAB to process human-readable text or log files. In this section we'll introduce a few of the common operations.

The `strfind` function searches for a pattern. The first argument is the string to be searched and the second is the pattern. It returns an array of indexes where it found the pattern. The pattern can be another string:

```
>> word = "Banana";
>> indexes = strfind(word, "a")
    indexes =
         2     4     6
>> indexes = strfind(word, "na")
    indexes =
          3     5
```

A similar function is `contains` which uses the same syntax, but returns a logical scalar if the pattern is found.

```
>> found = contains(word, "a")
found =
  logical
   1
>> found = contains(word, "b")
found =
  logical
   0
>> found = contains(word, "b", "IgnoreCase",true)
found =
  logical
   1
```

The `split` function breaks a string into an array of strings based on a *delimiter*, a character

or sequence of characters that separates multiple elements in the same string. Commas and backslashes are common delimiters in text data. GPS receives can report their information with standard NMEA text sentences that are comma delimited.

```
>> gpsheading = "$HCHDG,123.4,0.0,E,5.5,W*3C";
>> parts = split(gpsheading, ",")
parts =
  6x1 string array
    "$HCHDG"
    "123.4"
    "0.0"
    "E"
    "5.5"
    "W*3C"
```

We often want to convert these strings back into numerical values. For strings that are only numbers, the str2num function performs this operation. If the string can't be interpreted as a number the function returns and empty array. Continuing the GPS example above...

```
>> hdg = str2num(parts(2))
    hdg =
      123.4000
>> direction = str2num(parts(4))
    direction =
         []
```

1.3 Cell Arrays

So far we've seen arrays of numbers and arrays of strings. A cell array is a flexible data type where each element/cell can contain any type of data.

```
>> stra = ["hello", "world"];
>> vec = 5:-1:3;
>> num = 3.14;
>> carray = {stra, vec, num}
carray =
  1x3 cell array
    {["hello"    "world"]}    {[5 4 3]}    {[3.1400]}
```

The elements, or cells, are accessible by index, as with other MATLAB arrays, but in for cell arrays we use the curly braces {}. It is a little tricky when using an array of indexes to access multiple elements at one time.

```
    >> a = carray{1}
    a =
      1x2 string array
        "hello"    "world"
```

```
    >> [b, c]  = carray{1:2}
    b =
      1x2 string array
        "hello"    "world"
    c =
         5     4     3
```

1.4 Structures

Structures are a common type in MATLAB to store data in fields with a common name. We can create a single structure by using a period between the variable name and the field name the assigning it a value. The values of each field can be any data type.

```
>> rectangle.color = "red";
>> rectangle.width = 200;
>> rectangle.height = 100;
>> rectangle.corner = [0 0];
>> rectangle
rectangle =
  struct with fields:

     color: "red"
     width: 200
    height: 100
    corner: [0 0]
```

We can define an array of structures manually

```
    % Initialize a 2x1 structure array manually
    people(1).name = 'Alice';
    people(1).age = 25;
    people(2).name = 'Bob';
    people(2).age = 30;
```

or using the `struct` function

```
% Create a 2x1 structure array using struct
people = struct('name', {'Alice', 'Bob'}, 'age', {25, 30});
```

1.5 Exercises

Exercise 1.1. Numeric data types and formatting strings.

Create an m-file named `rdivision.m`. Within that script include a local function called `robust_division(a,b)` that calculates the quotient of two numbers, a/b. The function should be *fruitless*, without any return value. The function should print a single string. The

function should avoid integer division so that integer type inputs are converted to floating-point values (doubles). If the result is a `NaN`, print `"Not a Number"`. If the result is `Inf`, print `"Infinity"`. Otherwise, the function should print the result to the screen with three numbers to the right of the decimal point.

Within the same script include sufficient tests to verify the operation of the local function.